SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	Sasikumar Megha	**Reg. No.:**	192471038
Section / Batch:		**Date:**	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:		**100 Marks**

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I Sasi Kumar Megha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Megha

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)	20	28	19	19	10	46
Q2 (50 Marks)						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature:	Signature:	Signature:
Date: 20/6/27	Date:	Date:

CO1 AT1

Sasikumar Megha
192471038
CSA0601

Q91) Application : Online Learning Progress Tracker

A learning system processes student data using the recurrence $T(n) = 3T(n/3) + n$. Apply the master Theorem to determine the time complexity. Identify Parameter and determine the space complexity.

Online Learning Progress :

An online learning progress tracker analyzes students marks, attendence, completed lessons, quiz scores, and learning progress. The student data is divided into three equal groups. Each group is processed recursively, and the results are combined in linear time.

for example, the system may divide students into :

* High - performing students
* Average - performing students
* Students requiring improvement.

Algorithm Idea:

Learning Progress Tracker (data, n)

1. If n is small, process the student record direc
2. Divide the student data into three equal groups
3. Recursively process the first group.
4. Recursively process the second group
5. Recursively process the third group
6. combine the progress results in linear time.

Master Theorem Parameters:

Comparing with:

$$T(n) = aT(n/b) + f(n)$$

we get:

$$a = 3, \quad b = 3, \quad f(n) = n$$

calculate:

$$n^{\log_b a} = n^{\log_3 3} = n$$

Now

$$f(n) = (-)\ (n^{\log_b a})$$

This satisfies Master Theorem case 2

Time complexity :

$T(n) = (-)(n \log n)$

→ At each recusion level the total amount of work is 'n'

The number of recursion level is $\log_3 n$

$\therefore n \times \log_3 n = (-)(n \log n)$.

Space complexity :

The recusion depth is : $\log_3 n$

→ The auxilliary recursion - stack space is $O(\log n$

If a new array is created at each division the total memory may become $O(n)$. with index based division the auxiliary space is $O(\log n)$.

Efficiency and Order of Growth

* Time efficiency : $(-)(n \log n)$
* Auxiliary space efficiency : $O(\log n)$
* Order of growth : Linearithmic
* Efficiency level : efficient for processing large

student datasets.

Best, Average and worst cases

The recurrence performs the same number of division and the same combining work for every input.

* Best case : $\Theta(n \log n)$
* Average case : $\Theta(n \log n)$
* worst case : $\Theta(n \log n)$

(22) Application : Smart warehouse Inventory System

A warehouse system processes inventory using the recurrence $T(n) = 4T(n/2) + n \log n$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and analyze the space complexity.

Smart Warehouse Inventory System:

A smart warehouse inventory system processes Product quantities, stock levels, storage locations expiry dates and product demand.

The inventory is divided into four subproblem. Each subproblem contains half of the original input size. After recursively processing the groups, the system perform n log n additional work for sorting, searching, & combining the inventory information.

The system can be used for,

* Tracking available stock
* Identifying low-stock products
* sorting products according to demand
* Managing warehouse sections.
* Generating inventory reports.

Algorithm Idea :

Warehouse Inventory (data, n)

1. If n is small, process the inventory item directly
2. Create four subproblems, each of size n/2.
3. Recursively process all four subproblems.
4. Sort and combine the inventory results.
5. Generate the final stock report.

Master Theorem Parameters:

$a = 4$; $b = 2$; $f(n) = n \log n$

Calculate:

$n^{\log_b a} = n^{\log_2 4} = n^2$

compare

$f(n) = n \log n$

with:

$n^{\log_b a} = n^2$

since:

$n \log n = O(n^{2-\epsilon})$

for an appropriate $\epsilon > 0$; $f(n)$ is polynomially smaller than n^2.

Therefore, this satisfies Master Theorem case 1

Time Complexity:

$T(n) = (-)(n^2)$

The recursive subproblems contribute more computational work than the $n \log n$ combining operation. Therefore the recursive part dominates the time

complexity

Space complexity

At every recursive call; the input size is reduced form n to n/2.

The recursion depth is:

$\log_2 n$

Therefore, the recursion - stack space is $O(\log n)$

If the system creates separate arrays for inventory divisions, additional storage may be $O(n)$ or more. with in place processing, the auxiliary stack space is $O(\log n)$.

Efficiency and Order of Growth:

* Time efficiency : $(-)(n^2)$
* Auxiliary space efficiency : $O(\log n)$
* Order of growth : Quadratic
* Efficiency level : suitable for moderate inventory sizes, but slower for extremely large datasets.

Best, Average and worst cases

* Best case : $(-)(n^2)$
* Average case : $(-)(n^2)$
* worst case : $(-)(n^2)$

(Q93) Digital Image Filtering System

An image processing system follows the recurrence $T(n) = 3T(n/3) + n$. Apply the Master Theorem to determine the time complexity. Identify parameters and determine the space complexity.

A digital image filtering system divides an image into 4 regions or quadrants. Each quadrant is recursively processed to perform operations such as.

* Noise removal
* Image sharpening
* Blur correction

After processing the four regions, the system Perform n^2 work to combine or filter all the pixels.

Algorithm Idea:

Imagefilter (image, n)

1. If the image region is very small, filter it directly.
2. Divide the image into four region
3. Recursively filter the top-left, top-right region.
4. Recursively filter the botton-left, bottom-right region

Master Theorem Parameters:

$a = 4$, $b = 2$, $f(n) = n^2$

$n^{\log_b a} = n^{\log_2 4} = n^2$

$f(n) = (-)n^2$

$f(n) = (-)(n^{\log_b a})$

This satisfies Master Theorem case 2

Efficiency and Order of Growth

* Time efficiency : $(-)(n^2 \log n)$
* Auxilliary stack space : $O(\log n)$
* Total space with image data : $O(n^2)$
* Order of growth : Quadratic - Logarithmic
* efficiency level : Suitable for recursive image Processing, but expensive high-resolution image.

Best, Average and worst cases

* Best case : $(\cdot)(n^2 \log n)$
* Average case : $(\cdot)(n^2 \log n)$
* worst case : $(\cdot)(n^2 \log n)$

(Q94) Fraud detection in financial system

A fraud detection system processes transactions using the recurrence $T(n) = T(n-1) + n^2$. Solve the recurrence using substition method, and determine the time complexity. Also analyse the space complexity.

A fraud-detection system examines financial transactions one by one. for each set of n transactions. the system analyze one transaction and compares its properties with previous transaction patterns.

The n^2 work may represent:

* comparing transaction pairs
* checking customer behaviour
* Detecting unusual money transfers
* calculating fraud-risk scores.

Algorithm Idea:

Frauddetection (transaction, n)

1. If $n=1$, verify the transaction directly.
2. Rewrisively process the first $n-1$ transaction.
3. compare transaction patterns using n^2 operations.
4. Mark suspicious transactions.
5. Return the fraud-detection result.

Substition Method:

$$T(n) = T(n-1) + n^2$$

$$T(n-1) = T(n-2) + (n-1)^2$$

$$T(n) = T(n-2) + (n-1)^2 + n^2$$

$$T(n) = T(n-3) + (n-2)^2 + (n-1)^2 + n^2$$

$$T(n) = T(1) + 2^2 + 3^2 + \dots n^2$$

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$T(n) = T(1) + \frac{n(n+1)(2n+1)}{6}$$

$$\frac{n(n+1)(2n+1)}{6} = \frac{2n^3 + 3n^2 + n}{6}$$

Time Complexity

$$T(n) = \Theta(n^3)$$

Space Complexity:

The recursion-stack space is : $O(n)$

Efficiency and Order of Growth:

* Time efficiency : $\Theta(n^3)$
* Recursive space efficiency : $O(n)$
* Iterative auxiliary space : $O(1)$
* Order of growth : cubic

Best, Average and Worst cases

* Best case : $\Theta(n^3)$
* Average case : $\Theta(n^3)$
* worst case : $\Theta(n^3)$

Q95) Online Search Suggestion System:

A search system processes queries using the recurrence $T(n) = T(n/2) + n$. Solve the recurrence and determine the time complexity. Also determine the space complexity.

An online search-suggestion system analyses user-entered text and generates relevant suggestions. At every step, the system reduces the search space by half and performs linear work to rank or filter the available suggestion.

Algorithm Idea :

Searchsuggestion (data ,n)

1. If only one suggestion remains, return it
2. Reduce the search space to half.
3. Recursirely process the selected half.
4. Scan and rank the current suggestion in linear time
5. Returns the most relavent suggestions.

Master Theorem Parameters :

$a = 1$, $b = 2$, $f(n) = n$

$n\log_b{}^a = n\log_2{}^1 = n^0 = 1$

$f(n) = n$

$af(n/b) = 1(\frac{n}{2}) = \frac{n}{2}$

$c = 1/2 < 1$;

$af(n/b) \leq cf(n)$

$T(n) = (-)(f(n))$

Time complexity :

$T(n) = (-)(n)$

$T(n) = T(n/2) + n$

$T(n) = T(n/4) + \frac{n}{2} + n$

$T(n) = T(1) + n + \frac{n}{2} + \frac{n}{4} + \cdots$

$n + \frac{n}{2} + \frac{n}{4} + \cdots < 2n$

$T(n) = (-)(n)$

Space complexity :

$n, \frac{n}{2}, \frac{n}{4}, \ldots$

$\log_2 n$

$O(\log n)$

If implemented iteratively, the auxiliary space can be reduced to $O(1)$

Efficiency and Order of Growth

* Time efficiency : $\Theta(n)$
* Recursive auxiliary space : $O(\log n)$
* Iterative auxiliary space : $O(1)$
* Order of growth : linear

Best, Average and worst cases

* Best case : $\Omega(n)$
* Average case : $\Theta(n)$
* worst case : $O(n)$
*